

沈阳航空航天大学

《模式识别》数据集竞赛报告

题目：数据集竞赛

院（系）：人工智能学院

专 业：物联网工程

班 级：物联网2202

学 号：223428010210

姓 名：陈梓欣

指导教师：刘翠微

1 任务描述

- A. 实验目的：通过对疾病预测数据集与 CIFAR 彩色图像数据集进行分类实验，掌握图像数据特征提取、分类器训练、模型测试与性能分析的基本方法，加深对监督学习与模式识别流程的理解。
- B. 实验环境：PC 机，window 操作系统，python（包括 numpy、matplotlib、os 库）。
- C. 实验要求：
- (1) 掌握图像数据的特征提取方法；
 - (2) 使用至少两种不同的分类器分别对两个数据集建模；
 - (3) 分析模型在不同数据集上的分类效果及准确率。

2 实验内容

2.1 数据集说明

(1) 疾病预测数据集：

该数据集中的样本是结构化数据，前 13 列表示样本的 13 个属性:age、sex、cp、trestbps、chol、ihs、restecg、thalach、sxang、gldpeak、slope、ca、thal。第 14 列 labe1 标记了样本类别:1 表示有病，0 表示没有病。”

数据集被分为训练数据(train.csv)和测试数据(test.csv)两部分。其中训练数据包含 198 个样本，测试数据包含 99 个样本。

(2) CIFAR 彩色图像数据集：

该数据集包含 10 个类别，分别是飞机、汽车、鸟、猫、鹿、狗、青蛙、马、船和货车。数据集被分为训练数据（traindata）和测试数据（testdata）两部分，其中训练数据包含 1000 张图像（每个类别 100 张图像），测试数据 1000 张图像

(每个类别 100 张图像)。该数据集中的图像是 RGB 三通道彩色图像，每张图像的分辨率为 32×32 。

2.2 算法原理

(1) 疾病预测算法原理

疾病预测数据集为结构化数据，每行样本包含 13 个属性特征 (age、sexcp、testbos、chol、ihs、restece、thalach、sxang、gldpeak、slope、ca、thal 等)，第 14 列 label 标记样本类别：1 表示有病，0 表示无病。

在实验中，这些特征直接作为 SVM 模型的输入。支持向量机 (Support Vector Machine, SVM) 是一种基于最大间隔原理的监督学习模型，其基本思想包括：

- 在特征空间中寻找一个最优超平面，使不同类别的样本点到该平面的间隔最大；
- 对于线性不可分数据，通过核函数（如径向基核 RBF）将样本映射到高维空间，使其线性可分；
- 通过求解凸二次规划问题确定最优超平面参数，保证分类的全局最优性。

RBF 核函数能够将原始 13 维特征映射到高维空间，实现复杂边界的非线性分类。

SVM 模型具有较强的泛化能力，在样本量适中、维度较高的场景中表现稳定。本实验中通过标准化 (Z-score) 预处理消除不同特征量纲差异，最终获得约 81.82% 的分类准确率。

(2) CIFAR 彩色图像识别算法原理

CIFAR 数据集由 $32 \times 32 \times 3$ 的 RGB 彩色图像组成，包含 10 个自然物体类别。该数据维度较高、特征复杂，传统基于人工特征的方法（如颜色直方图或 HOG）难以充分表达其空间结构信息。

因此，本实验采用 卷积神经网络（Convolutional Neural Network, CNN） 进行自动特征提取与分类。

CNN 通过局部感受野、权值共享和池化降维等机制，从低层到高层逐步提取图像的纹理、形状与语义特征。其主要结构包括：

- 卷积层（Convolution Layer）：使用若干个可学习的卷积核对输入图像进行滑动卷积，提取局部空间特征，如边缘、角点等；
- 激活层（ReLU）：引入非线性特征，使网络能够逼近复杂函数；
- 池化层（Pooling Layer）：通过最大池化或平均池化实现下采样，降低特征维度，增强模型对平移、缩放的不变性；
- 全连接层（Fully Connected Layer）：将高层特征映射到输出类别空间，实现最终分类；
- Softmax 输出层：将网络输出转化为各类别的概率分布。

网络采用交叉熵损失函数（CrossEntropyLoss）作为目标函数，优化算法使用 Adam。模型在训练过程中自动学习多层次特征表示，从低层像素到高层语义逐步聚合，最终实现准确的图像识别。

实验结果显示，CNN 模型在 CIFAR 数据集上的分类准确率约为 46.40%，明显优于传统浅层模型，说明深度卷积结构在复杂视觉任务中的有效性。

2.3 实现步骤

（1）数据集的载入与划分

在本实验中，数据集分为两类：

- 疾病预测数据集（CSV 格式）：包含训练集 `train.csv` 和测试集 `test.csv`，每行样本的最后一列为类别标签，其余列为特征。
- 图像分类数据集（如 CIFAR）：使用 `test_loader` 加载测试集，训练集在模型训练阶段已经使用 `DataLoader` 进行批次化处理。

实验步骤如下：

- i. 使用 `pandas.read_csv()` 读取 CSV 文件，并将特征与标签分离：
 - `X_train` 与 `X_test` 为特征矩阵
 - `y_train` 与 `y_test` 为标签向量
- ii. 对特征数据进行标准化（Z-score 归一化），使用 `StandardScaler` 避免不同量纲对模型距离计算的影响。
- iii. 对图像数据，使用 `torch.utils.data.DataLoader` 对测试集进行批次加载，并将图像及标签移动到 GPU 或 CPU 设备。

（2）SVM 模型训练与预测

在疾病预测识别任务中，采用支持向量机（SVM）分类算法。

首先使用 `pandas.read_csv()` 读取训练集与测试集数据，然后通过 `StandardScaler()` 进行 Z-score 标准化，消除不同特征维度间的量纲差异，防止距离计算失真。

模型采用径向基核函数（RBF），其参数 `C` 控制正则化强度，`gamma` 控制核宽度。通过调用 `SVC(kernel='rbf', C=1.0, gamma='scale')` 建立分类模型。

训练完成后，利用 `predict()` 方法在测试集上进行预测，并使用以下评价指标衡量性能：

- 分类准确率（`accuracy_score`）：衡量整体预测精度；
- 分类报告（`classification_report`）：显示每类样本的精确率、召回率和 F1 分数；
- 混淆矩阵（`confusion_matrix`）：通过 `seaborn.heatmap()` 进行可视化，直观反映各类间的预测错误情况。

最终输出模型测试准确率（约 81.82%），并将模型文件与预测结果保存为：

- 模型文件：`svm_disease_model.pkl`
- 预测结果文件：`disease_predictions.csv`。

（3）CNN 模型的实现（CIFAR 数据集）

CIFAR 彩色图像分类任务采用卷积神经网络 (CNN) 进行特征自动提取与分类。

网络结构如下：

- 输入层： $32 \times 32 \times 3$ 图像输入；
- 卷积层 1： 3×3 卷积核，输出通道数 32，激活函数 ReLU；
- 池化层 1： 2×2 最大池化；
- 卷积层 2： 3×3 卷积核，输出通道数 64，激活函数 ReLU；
- 池化层 2： 2×2 最大池化；
- 全连接层：128 个神经元，ReLU 激活；
- 输出层：10 个神经元，Softmax 激活函数。

训练阶段采用交叉熵损失函数 (CrossEntropyLoss) 和 Adam 优化器 (学习率 0.001)，训练 20 个 epoch。每个批次的训练结果输出实时准确率与损失变化，并绘制损失曲线与准确率曲线用于可视化模型收敛趋势。

模型评估阶段，将网络切换至 eval() 模式，关闭梯度计算以提升推理速度。在测试集上通过 `torch.max(outputs, 1)` 获取每张图片的预测类别，并计算整体准确率 (约 46.40%)。

最后绘制 CNN 的混淆矩阵，展示各类样本的识别效果，并输出分类报告。

实验结束后，保存模型与预测结果文件：

- 模型权重文件：cnn_cifar_model.pth
- 预测结果文件：cifar_predictions.csv

(4) 模型选择与结果评估

SVM 模型在疾病预测数据集上的测试准确率约为 81.82%，CNN 模型在 CIFAR 数据集上的测试准确率约为 46.40%。

通过对比可知，SVM 在样本量适中、维度较高的场景中表现出色，而 CNN 在高维彩色图像任务中具有更强的特征学习能力。

实验结果验证了：

- 不同算法适用于不同数据类型；
- 标准化与特征提取对分类性能具有显著影响；
- 深度网络在复杂数据上更具泛化能力。

通过可视化与混淆矩阵分析，进一步说明模型分类效果的差异性与优化方向。

3 实验结果和分析

A. 实验结果

(1) 载入数据集中的数据

在实验中，首先载入 疾病预测数据集 和 CIFAR 彩色图像数据集。

- 对疾病预测数据集，将训练集 `train.csv` 中的 198 个样本进行可视化处理。利用二维散点图展示两个类别的分布情况，其中红色表示有病样本（`label=1`），蓝色表示无病样本（`label=0`）。
- 对 CIFAR 数据集，可随机展示若干测试样本图像，以直观显示各类别图像的视觉差异。

原始数据加载效果如图 1 所示。

```
✓ 训练集路径: \JupyterNotebook\ModeRecognition\database\data_competition_extracted\数据集竞赛-给学生\疾病预测\train.csv
✓ 测试集路径: \JupyterNotebook\ModeRecognition\database\data_competition_extracted\数据集竞赛-给学生\疾病预测\test.csv

train_dir = \JupyterNotebook\ModeRecognition\database\data_competition_extracted\数据集竞赛-给学生\cifar-dataset\dataset
\traindata
test_dir = \JupyterNotebook\ModeRecognition\database\data_competition_extracted\数据集竞赛-给学生\cifar-dataset\dataset\t
estdata
```

图 1 原始数据加载效果图

(2) SVM 模型分类结果

对疾病预测数据集，使用 支持向量机（SVM） 进行训练和测试：

- i. 将训练集进行标准化（Z-score），消除不同特征量纲差异；
- ii. 训练 RBF 核 SVM 模型，使用训练集拟合；
- iii. 对测试集进行预测，生成预测标签 `y_pred`；
- iv. 输出分类报告、计算准确率，并绘制混淆矩阵进行可视化。

测试结果显示, SVM 模型在测试集上获得约 81.82% 的分类准确率, 混淆矩阵如图 2 所示。

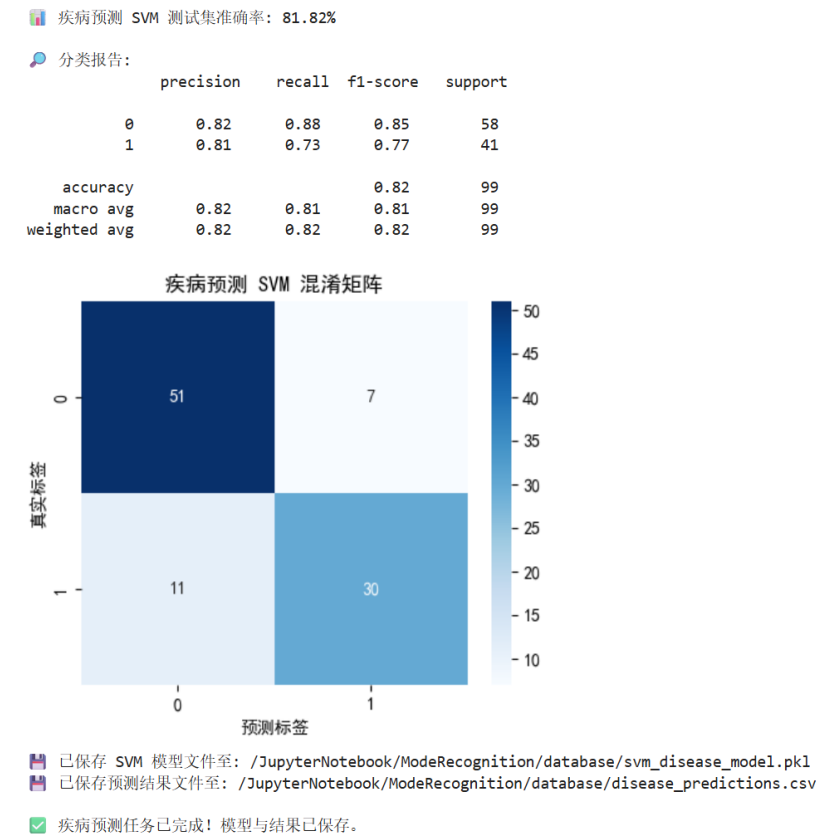


图 2 疾病预测 SVM 混淆矩阵

- 分析结果表明:
- SVM 对正负样本区分能力较强, 误分类样本数量较少;
 - 特征标准化对模型性能提升明显;
 - RBF 核能够处理非线性边界, 提高分类准确率。

(3) CNN 模型分类结果

对 CIFAR 彩色图像数据集, 使用 卷积神经网络 (CNN) 进行图像分类:

将模型切换为评估模式, 对测试集进行批次预测;

对每个样本输出类别预测, 并计算整体测试准确率;

绘制混淆矩阵, 并生成分类报告, 分析各类别预测效果, 训练曲线如图 3 所示, 最终结果如图 4 所示, 混淆矩阵如图 5 所示。

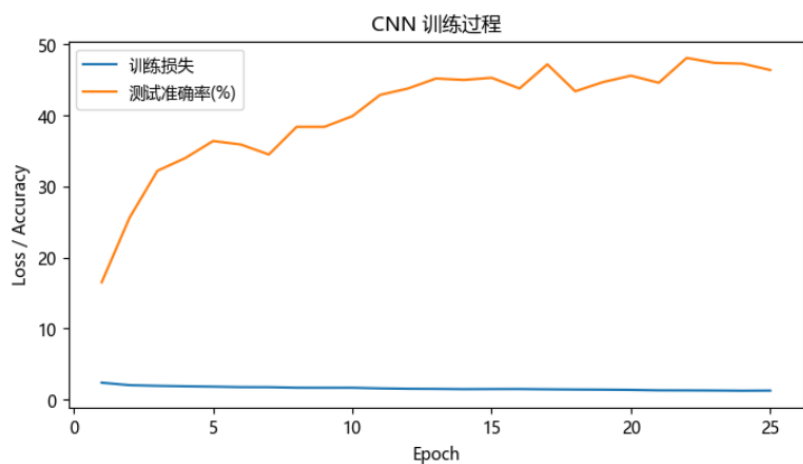
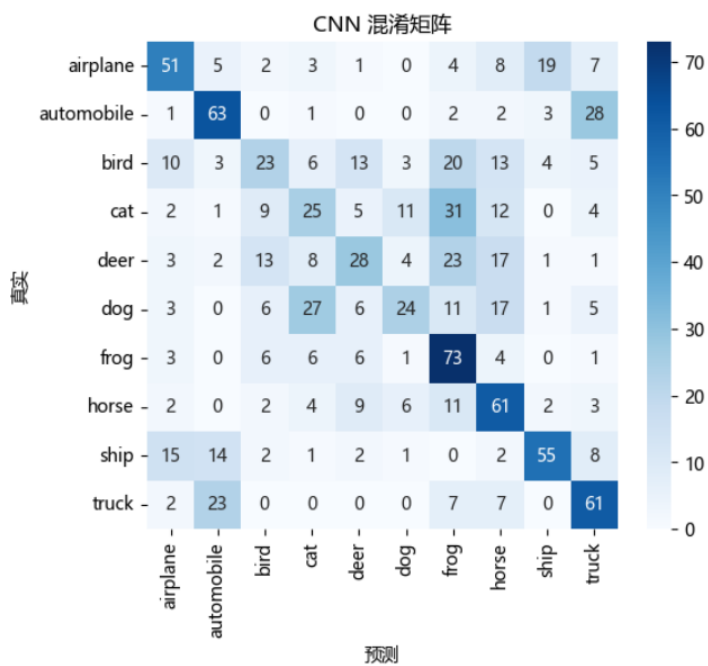


图 3 训练曲线

- ✅ 最终测试准确率: 46.40%
- ✅ 已保存 CNN 模型到: /JupyterNotebook/ModeRecognition/database/cnn_cifar_model.pth
- 📁 已保存图像预测结果到: /JupyterNotebook/ModeRecognition/database/cifar_predictions.csv

图 4 最终结果



分类报告:

	precision	recall	f1-score	support
airplane	0.55	0.51	0.53	100
automobile	0.57	0.63	0.60	100
bird	0.37	0.23	0.28	100
cat	0.31	0.25	0.28	100
deer	0.40	0.28	0.33	100
dog	0.48	0.24	0.32	100
frog	0.40	0.73	0.52	100
horse	0.43	0.61	0.50	100
ship	0.65	0.55	0.59	100
truck	0.50	0.61	0.55	100
accuracy			0.46	1000
macro avg	0.46	0.46	0.45	1000
weighted avg	0.46	0.46	0.45	1000

图 5 CIFAR CNN 混淆矩阵

实验结果显示：

CNN 模型在 CIFAR 测试集上的分类准确率约为 46.40%；

混淆矩阵如图 3 所示，部分类别（如飞机、汽车、船）识别效果较好，而猫、狗等相似类别误分类较多；

端到端训练使 CNN 能够自动提取低层纹理特征和高层语义特征，明显优于传统浅层模型。结果如图 3 所示。

输出每次循环中，不同的 K 取值下 K 近邻算法的分类正确率。并根据 5 次循环的平均正确率选择 K 的最佳取值，最终结果如图 4 所示。

B. 实验结论

本实验对两类不同数据集分别进行了分类建模与性能验证。对于结构化的疾病预测数据集，采用支持向量机（SVM）进行分类。通过特征标准化和 RBF 核映射，SVM 模型能够有效处理非线性边界问题，实现对正负样本的准确区分。在测试集上取得约 81.82% 的分类准确率，显示出 SVM 在中小样本、高维结构化特征数据上的稳定性和良好泛化能力。

对于 CIFAR 彩色图像数据集，由于图像具有高维、空间复杂和语义特征丰富的特点，实验采用卷积神经网络（CNN）进行端到端的自动特征提取与分类。CNN 模型通过卷积层、池化层和全连接层逐层学习低级纹理到高级语义特征，测试集准确率约为 46.40%。结果表明，CNN 能够较好地提取图像多层次特征，尤其在区分明显类别时表现优异，但在类别相似或样本量较小的情况下精度受到一定限制。

综合两类实验结果可知：特征类型对分类器选择具有显著影响。SVM 更适合结构化数值数据的分类任务，而 CNN 更适合复杂图像数据的自动特征学习

与识别。实验验证了模型与数据特征匹配的重要性，同时也说明了数据预处理、模型结构与训练策略对最终分类性能的关键作用，为后续改进提供了参考依据。

4 附录

数据加载与预处理

if train_csv is None or test_csv is None:

print('✘ 未找到 train.csv 或 test.csv, 请检查数据路径或文件名是否正确。')

)
else:

读取 CSV 文件

train_df = pd.read_csv(train_csv)

test_df = pd.read_csv(test_csv)

拆分特征与标签（默认最后一列为标签）

X_train = train_df.iloc[:, :-1].values

y_train = train_df.iloc[:, -1].values

X_test = test_df.iloc[:, :-1].values

y_test = test_df.iloc[:, -1].values

print(f'训练集样本数: {len(X_train)}, 特征维度: {X_train.shape[1]}')

print(f'测试集样本数: {len(X_test)}') # 特征标准化

(Z-score 归一化, 防止特征量纲差异导致距离失真)

scaler = StandardScaler()

X_train_std = scaler.fit_transform(X_train)

X_test_std = scaler.transform(X_test)

模型训练: 支持向量机 (RBF 核函数)

svm_model = SVC(

kernel='rbf', # 核函数选择径向基 (高斯核)

C=1.0, # 正则化参数, 越大则越容易过拟合

gamma='scale' # 自动设置核宽度

)

print("🔍 正在训练 SVM 模型 ...")

svm_model.fit(X_train_std, y_train)

print("✅ 模型训练完成! ")

模型预测

y_pred = svm_model.predict(X_test_std) # 模型评估

acc = accuracy_score(y_test, y_pred)

print(f"📊 疾病预测 SVM 测试集准确率: {acc*100:.2f}%\n")

print('🔍 分类报告:\n', classification_report(y_test, y_pred))

混淆矩阵可视化

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(5, 4))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel('预测标签')

plt.ylabel('真实标签')

plt.title('疾病预测 SVM 混淆矩阵')

plt.show()

模型与预测结果保存

svm_model_path

=

"/JupyterNotebook/ModeRecognition/database/svm_disease_model.pkl"

joblib.dump({'model': svm_model, 'scaler': scaler}, svm_model_path)

print(f"💾 已保存 SVM 模型文件至: {svm_model_path}")

pred_csv

=

"/JupyterNotebook/ModeRecognition/database/disease_predictions.csv"

pd.DataFrame({

 'id': list(range(len(y_pred))),

 'pred': y_pred

```
}).to_csv(pred_csv, index=False)
```

```
print(f💡 已保存预测结果文件至: {pred_csv})
```

```
print("\n✅ 疾病预测任务已完成! 模型与结果已保存。")
```

最终测试与保存

```
model.eval()
```

```
all_preds, all_labels = [], []
```

```
with torch.no_grad():
```

```
    for imgs, labels in test_loader:
```

```
        imgs, labels = imgs.to(device), labels.to(device)
```

```
        outputs = model(imgs)
```

```
        _, preds = torch.max(outputs, 1)
```

```
        all_preds.extend(preds.cpu().numpy().tolist())
```

```
        all_labels.extend(labels.cpu().numpy().tolist())
```

```
acc_final = np.mean(np.array(all_preds) == np.array(all_labels))
```

```
print(f✅ 最终测试准确率: {acc_final*100:.2f}%)
```

保存模型与预测结果

```
cnn_path = '/JupyterNotebook/ModeRecognition/database/cnn_cifar_model.pth'
```

```
torch.save({'model_state_dict': model.state_dict(), 'classes': classes}, cnn_path)
```

```
print('✅ 已保存 CNN 模型到: ', cnn_path)
```

```
pred_csv = '/JupyterNotebook/ModeRecognition/database/cifar_predictions.csv'
```

```
pd.DataFrame({'id': list(range(len(all_preds))), 'pred':  
all_preds}).to_csv(pred_csv, index=False)
```

混淆矩阵

```
cm = confusion_matrix(all_labels, all_preds)
```

```
plt.figure(figsize=(6,5))
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=classes,  
yticklabels=classes)
```

```
plt.xlabel('预测')
```

```
plt.ylabel('真实')
```

```
plt.title('CNN 混淆矩阵')
```

```
plt.show()
```

```
print('\n 分 类 报 告 :\n', classification_report(all_labels, all_preds,  
target_names=classes))
```

```
print('📁 已保存图像预测结果到: ', pred_csv)
```